

NFC-Based Mutual Authentication Protocol for Smart Lock Access Control

Technical Specification

Version 1.0 – Draft

July 2, 2026

Abstract

This document specifies a mutual authentication and secure session establishment protocol between a mobile application (“Phone”) and an ESP32-based smart lock controller (“Lock”) over NFC. The protocol combines long-term Ed25519 identities, ephemeral X25519 key agreement, a three-message SIGMA-style handshake, HKDF-SHA256 key derivation, and AES-256-GCM authenticated encryption to provide mutual device authentication, confidentiality, integrity, and forward secrecy for each NFC session. This revision addresses a signature-binding gap identified in the prior two-message design and documents current scope limitations and deferred work.

Contents

1	Scope and Design Assumptions	2
1.1	Non-Goals	2
2	System Architecture	2
2.1	Components	2
2.2	Cryptographic Libraries	3
3	Long-Term Identity	3
4	Handshake Protocol	3
4.1	Rationale for the Three-Message Design	3
4.2	Message Flow	4
4.3	Message Definitions	4
4.4	Verification Rules	4
5	Shared Secret and Key Derivation	5
5.1	Shared Secret	5
5.2	Key Derivation	5
6	Secure Communication	5
6.1	Message Format	5
6.2	Nonce Strategy for Low-Volume Sessions	6
6.3	Replay Protection Within a Session	6

7	Session Termination	6
8	Security Properties	6
9	Explicitly Deferred Items	7
9.1	ATECC608A Secure Element Migration	7
9.2	Authorization Model	7
9.3	Relay Attack Resistance	8
10	Complete Protocol Summary	8
11	Revision History	9

1 Scope and Design Assumptions

This specification governs cryptographic session establishment and secure messaging between the Phone and the Lock. The following scope decisions are in effect for this revision and are treated as explicit, reviewed trade-offs rather than oversights.

- **Handshake structure:** Three-message mutual authentication (upgraded from the earlier two-message design) to eliminate signature ambiguity. See Section 4.1.
- **Secure element:** The ESP32 currently stores the long-term Ed25519 private key in software (libsodium, RAM/flash). Migration to an ATECC608A secure element is planned; see Section 9.1.
- **Session message volume:** Each NFC session is expected to carry a low number of application messages (tens, not thousands). This justifies the use of randomly generated AES-GCM nonces without a counter construction; see Section 6.2.
- **Authorization model:** Authentication is currently treated as equivalent to authorization – any device that can prove possession of a provisioned Ed25519 identity key is granted access. No access control list, role, or revocation mechanism is defined in this revision. See Section 9.
- **Relay attacks:** Out of scope for this revision. The protocol does not attempt to bound proximity or handshake latency. See Section 9.

1.1 Non-Goals

This specification does not cover: provisioning workflows (manufacturing, QR-code pairing, onboarding UX), firmware update security, physical tamper response, power analysis / side-channel resistance of the current software Ed25519 implementation, or transport-layer framing below the NFC link layer (handled by `flutter_nfc_kit` / `nfc_manager` and the ESP-IDF NFC stack).

2 System Architecture

2.1 Components

Component	Platform	Role
Mobile Application	Flutter (iOS/Android)	Initiates NFC session, performs handshake, sends encrypted unlock commands
Lock Controller	ESP32 (ESP-IDF)	Verifies phone identity, performs handshake, actuates lock mechanism on authenticated command

2.2 Cryptographic Libraries

Platform	Library	Responsibility
Flutter	<code>cryptography</code>	Ed25519 sign/verify, X25519 agreement, HKDF-SHA256, AES-256-GCM, secure RNG
Flutter	<code>flutter_secure_storage</code>	Storage of Ed25519 private identity key (Keychain / Keystore)
Flutter	<code>flutter_nfc_kit</code> / <code>nfc_manager</code>	NFC transport only – no cryptographic operations
ESP32	<code>libsodium</code>	Ed25519 sign/verify, X25519 agreement, secure RNG; key storage today, ATECC608A-backed in future revision
ESP32	mbedTLS (ESP-IDF)	HKDF-SHA256, AES-256-GCM

No custom cryptographic primitives are implemented on either platform.

3 Long-Term Identity

Each device holds a permanent Ed25519 identity key pair, generated once at provisioning time.

Phone:	Ed25519 Private Key (Keychain / Keystore)
	Ed25519 Public Key (provisioned to Lock)
Lock:	Ed25519 Private Key (libsodium; ATECC608A in future revision)
	Ed25519 Public Key (provisioned to Phone / backend)

Public key distribution (manufacturing, QR-code pairing, onboarding) is governed by the trust model and is outside the scope of this document. This specification assumes both parties already hold an authentic copy of the peer’s Ed25519 public key before the handshake begins.

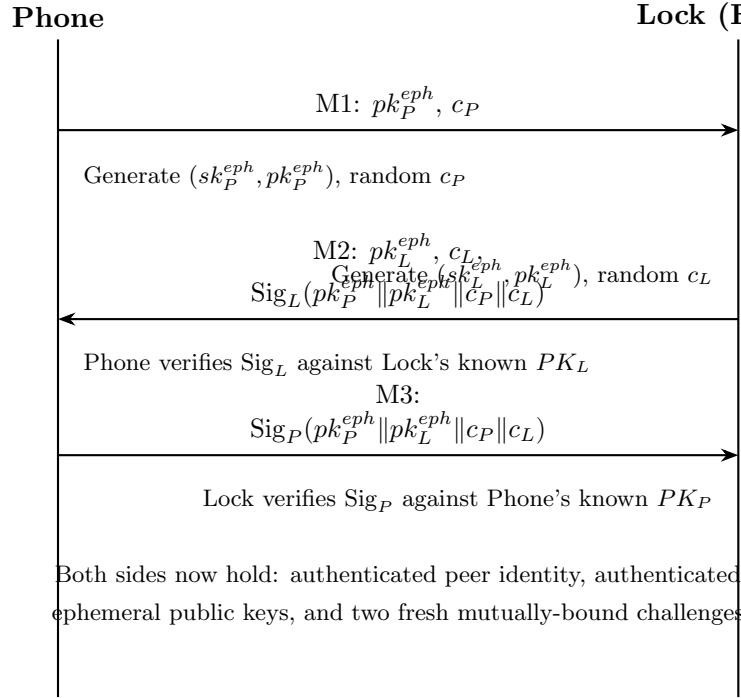
4 Handshake Protocol

4.1 Rationale for the Three-Message Design

The original two-message design had the Phone sign “the ephemeral public key and both challenges,” but the Phone’s first message is sent before it has seen the Lock’s challenge. A signature that does not cover the Lock’s challenge is not bound to the Lock’s contribution to the session, which permits splicing a valid Phone signature from one session into another session carrying a different Lock challenge.

This revision adopts a SIGMA-style three-message handshake: the Lock speaks after the Phone and signs both parties’ values, and the Phone closes the handshake with a final signature that also covers both parties’ values. Every signature produced by either party covers *both* ephemeral public keys and *both* challenges, so no signature is valid outside the session it was created for.

4.2 Message Flow



4.3 Message Definitions

Msg	Sender	Contents
M1	Phone	pk_P^{eph} (X25519, 32 bytes), c_P (random challenge, 32 bytes)
M2	Lock	pk_L^{eph} (X25519, 32 bytes), c_L (random challenge, 32 bytes), $Sig_L(pk_P^{eph} pk_L^{eph} c_P c_L)$ (Ed25519, 64 bytes)
M3	Phone	$Sig_P(pk_P^{eph} pk_L^{eph} c_P c_L)$ (Ed25519, 64 bytes)

Concatenation order $(pk_P^{eph} || pk_L^{eph} || c_P || c_L)$ is fixed and identical for both signatures, and **must** be implemented identically on both platforms. A protocol version byte and a domain separation prefix (e.g. "SLOCK-HS-v1") SHOULD be prepended to the signed payload on both sides to prevent cross-protocol signature reuse.

4.4 Verification Rules

1. Upon receiving M2, the Phone **MUST** verify Sig_L using the Lock's provisioned long-term public key PK_L before proceeding to M3. On failure, the Phone **MUST** abort the session and **MUST NOT** derive any session key material.

2. Upon receiving M3, the Lock MUST verify Sig_P using the Phone's provisioned long-term public key PK_P before treating the session as authenticated. On failure, the Lock MUST abort and MUST NOT actuate the lock mechanism for this session.
3. Both parties MUST reject a handshake if pk_P^{eph} or pk_L^{eph} is the all-zero point or otherwise fails X25519 validity checks (contributory behavior check / small-subgroup rejection, as provided by libsodium and the `cryptography` package by default).
4. Challenges c_P , c_L MUST be generated with a CSPRNG and MUST NOT be reused across sessions.

5 Shared Secret and Key Derivation

5.1 Shared Secret

Both devices independently compute the X25519 shared secret:

```
SharedSecret = X25519(local_ephemeral_private, remote_ephemeral_public)
```

No key material is transmitted during this step; each side derives the same value from its own ephemeral private key and the peer's ephemeral public key received during the handshake.

5.2 Key Derivation

The session key is derived via HKDF-SHA256, binding in both challenges so that the resulting key is cryptographically tied to this specific handshake instance:

```
PRK = HKDF-Extract(salt = c_P || c_L, ikm = SharedSecret)
Key = HKDF-Expand(PRK, info = "phone->esp" | "esp->phone", L = 32)
```

Two directional keys are derived using distinct `info` strings to prevent key reuse across communication directions:

Direction	HKDF info string
Phone → Lock	"phone->esp"
Lock → Phone	"esp->phone"

Each derived key is 32 bytes (256 bits), used directly as an AES-256-GCM key.

6 Secure Communication

6.1 Message Format

All application-layer messages after the handshake are encrypted with AES-256-GCM using the appropriate directional key:

Field	Size	Description
Nonce	12 bytes	CSPRNG-generated, unique per message per key (see Section 6.2)
Ciphertext	variable	AES-256-GCM output for the application payload
Auth Tag	16 bytes	GCM authentication tag

6.2 Nonce Strategy for Low-Volume Sessions

Given the low expected number of messages per session (typically a single unlock command and acknowledgment, rarely exceeding a few dozen messages), random 96-bit nonces are considered acceptable for this revision. The birthday-bound collision risk for random GCM nonces becomes significant only as message counts approach the range of 2^{32} under a single key; the per-session ephemeral key lifetime and low message volume keep the collision probability negligible.

Recommendation for future hardening: should session message volume increase substantially (e.g. streaming telemetry, high-frequency polling), this SHOULD be revisited in favor of a deterministic counter-based nonce (optionally salted with a random per-session prefix) to remove reliance on RNG quality for nonce uniqueness.

6.3 Replay Protection Within a Session

Because a new session key is derived per NFC session, and each session is short-lived and single-purpose (e.g. one unlock command), sequence-number based replay protection within a session is currently OPTIONAL. If a session is extended to carry multiple sensitive commands, an incrementing sequence number SHOULD be included in the associated data (AD) passed to AES-GCM.

7 Session Termination

Upon completion or abort of the NFC session, both parties MUST securely erase the following from memory:

- Ephemeral X25519 private key (sk^{eph})
- X25519 shared secret
- HKDF intermediate output (PRK)
- Derived AES-256 session keys (both directions)

Only the long-term Ed25519 identity key persists between sessions. Each subsequent NFC interaction executes the full three-message handshake with freshly generated ephemeral keys and challenges; no session state is resumed or cached.

8 Security Properties

Property	Mechanism	Notes
Mutual Device Authentication	Ed25519 signatures over transcript ($pk_P^{eph}, pk_L^{eph}, c_P, c_L$)	Both signatures now cover the full transcript, closing the M1-only binding gap of the prior design
Confidentiality	AES-256-GCM	Directional keys via HKDF info separation
Integrity	AES-GCM authentication tag	16-byte tag per message
Key Agreement	X25519	Ephemeral, per-session
Key Derivation	HKDF-SHA256	Challenges bound into salt

Property	Mechanism	Notes
Replay Protection (handshake)	Fresh challenges in signed transcript	Prevents cross-session signature reuse
Forward Secrecy	Fresh X25519 pair per session, destroyed at termination	Compromise of long-term key does not expose past session traffic
Long-Term Key Protection (Phone)	OS / Keychain / Keystore via <code>flutter_secure_storage</code>	
Long-Term Key Protection (Lock)	Software (libsodium) in this revision; hardware secure element planned	See Section 9.1
Authorization	<i>Not yet implemented – see below</i>	Any authenticated identity is currently authorized
Relay Attack Resistance	<i>Not yet implemented – see below</i>	Out of scope this revision

9 Explicitly Deferred Items

The following items are acknowledged gaps, intentionally deferred rather than overlooked, and should be tracked for future revisions of this specification.

9.1 ATECC608A Secure Element Migration

The ESP32 currently manages its long-term Ed25519 private key in software via libsodium. A future revision will migrate long-term key storage and signing operations to the Microchip ATECC608A secure element, which provides:

- Hardware-isolated private key storage (key never leaves the secure element)
- Hardware-accelerated ECDSA (note: ATECC608A natively supports P-256 ECDSA, not Ed25519 – migration will require either (a) switching the long-term identity scheme to ECDSA P-256, or (b) using the ATECC608A purely as a protected key vault / HMAC-based challenge engine while retaining software Ed25519, depending on final part variant and firmware support)
- Tamper and side-channel resistant signing
- A hardware root of trust for device provisioning

Action item: confirm target ATECC608A variant and its supported curve set before finalizing whether the long-term identity scheme remains Ed25519 or migrates to P-256, as this affects both firmware and the mobile verification code path.

9.2 Authorization Model

Authentication currently implies authorization. Future work should introduce:

- An access control list (ACL) of authorized Phone public keys per Lock, or a signed capability/-credential issued by a backend authority

- A revocation mechanism (e.g. denylist, short-lived signed credentials, or online revocation check) so a lost or decommissioned phone can be removed without re-provisioning the Lock's entire trust store
- Optional role/scope differentiation (e.g. owner vs. guest vs. time-limited access)

9.3 Relay Attack Resistance

This revision does not defend against relay attacks, in which an attacker relays the NFC exchange between a legitimate Phone and a legitimate Lock over a long-distance channel. Candidate mitigations for future revisions include:

- Distance-bounding / round-trip time (RTT) measurement with a strict timeout on the M1→M2→M3 exchange
- Leveraging NFC's inherent short range together with a hard timeout as a first-order mitigation (does not fully prevent relay but raises attack cost)

10 Complete Protocol Summary

Phone	Lock (ESP32)
-----	-----
Load Ed25519 identity key	Load Ed25519 identity key
Generate X25519 ephemeral pair (pk_P, sk_P)	
Generate challenge c_P	
M1: pk_P, c_P ----->	
	Generate X25519 ephemeral pair (pk_L, sk_L)
	Generate challenge c_L
	Sig_L = Sign(pk_P pk_L c_P c_L)
	<----- M2: pk_L, c_L, Sig_L
Verify Sig_L using PK_L	
Sig_P = Sign(pk_P pk_L c_P c_L)	
M3: Sig_P ----->	
	Verify Sig_P using PK_P
[Both sides] SharedSecret = X25519(local_sk, remote_pk)	
[Both sides] PRK = HKDF-Extract(salt=c_P c_L, ikm=SharedSecret)	
[Both sides] K_p2e = HKDF-Expand(PRK, "phone->esp", 32)	
[Both sides] K_e2p = HKDF-Expand(PRK, "esp->phone", 32)	
<===== AES-256-GCM encrypted application messages =====>	
[Both sides] Destroy: sk_eph, SharedSecret, PRK, K_p2e, K_e2p	

11 Revision History

Version	Change	Date
0.1	Initial two-message handshake draft	—
1.0	Upgraded to three-message SIGMA-style handshake to close signature transcript binding gap; documented ATECC608A migration plan; scoped authorization and relay-attack mitigation as deferred work	July 2, 2026